Assignment 2: Building an RUDP Real-time Relay Server

Course: Game Networking and Server Development (GNS301)

Chapter 6 – High-Speed Transport (UDP / RUDP)

Type: Assignment (Individual)

Estimated Time: 3–5 hours

Total Points: 10.0

Learning Outcomes

By completing this assignment you will be able to:

- Integrate **LiteNetLib** (RUDP) into an existing ASP.NET Core project.
- Build a real-time relay server that handles multiple concurrent clients.
- Use different DeliveryMethod types appropriately for game data.
- Combine **REST API** (from Assignment 1) with **RUDP** in a single .NET application.
- Create a complete basic game backend that supports both persistent data (MongoDB + REST) and real-time synchronization.

This assignment builds directly on **Assignment 1**. You will extend the GameInventoryApi project to become a full-featured game server.

Prerequisites

- Completed **Assignment 1** (GameInventoryApi project with REST API and MongoDB)
- .NET 8 SDK
- Docker Desktop (MongoDB container from Assignment 1)
- Basic understanding of LiteNetLib (from Chapter 6 slides and Lab 6.2)

Assignment Overview

You will extend your existing GameInventoryApi project to include a **RUDP Real-time Relay Server** using LiteNetLib.

Final Architecture:

- **Port 5000 (HTTPS)** → REST API (Inventory, login, etc.)
- **Port 7777** → RUDP Real-time Relay Server (LiteNetLib)

The server will:

- Accept real-time connections from game clients

- Relay player positions / actions to all other connected clients
- Log connection/disconnection events
- (Optional advanced feature) Link RUDP connections with PlayerId from your existing MongoDB

TABLE OF CONTENT

Assignment 2: Building an RUDP Real-time Relay Server....1
PART 1: CREATE RUDP SERVER....3
Step 1: Open Your Existing Assignment 1 Project....3
Step 2: Add LiteNetLib Package....3
Step 3: Create Real-time Data Models....3
Step 4: Create RUDP Server Service....3
Step 5: Integrate RUDP Service into Program.cs....6
Step 6: Run and Test the Full Server....8
PART 2: CREATE TEST CLIENT APP....9
Step 1: Create the Test Client Project....9
Step 2: Replace the entire content of Program.cs....9
Step 3: Build and Run....12
Step 4: How to Test (Recommended for Assignment Submission)....12
Step 5: Screenshots for README.md (Submission)....13

PART 1: CREATE RUDP SERVER

Step 1: Open Your Existing Assignment 1 Project

Open the GameInventoryApi folder from Assignment 1 in VS Code or Visual Studio.

Step 2: Add LiteNetLib Package

Run the following command in the project root:

```
dotnet add package LiteNetLib --version 1.2.0
```

Step 3: Create Real-time Data Models

Create a new folder named **RealTime** and add the following file: PlayerState.cs

RealTime/PlayerState.cs

```
using System.Numerics;

namespace GameInventoryApi.RealTime;

public class PlayerState
{
    public string PlayerId { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public Vector3 Position { get; set; }
    public float RotationY { get; set; }
    public long Timestamp { get; set; } =
DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
}
```

Step 4: Create RUDP Server Service

Create a new folder named Services and add file RudpServerService.cs:

Services/RudpServerService.cs

```
using System;
using System.Numerics;
using LiteNetLib;
using LiteNetLib.Utils;
using GameInventoryApi.RealTime;
using System.Net;

namespace GameInventoryApi.Services;

public class RudpServerService
{
    private readonly EventBasedNetListener _listener;
    private readonly NetManager _manager;
    private readonly NetDataWriter _writer = new();

    public RudpServerService()
    {
        _listener = new EventBasedNetListener();
        _manager = new NetManager(_listener);

        _manager.UpdateTime = 15;

        // Đăng ký events
        _listener.PeerConnectedEvent += OnPeerConnected;
        _listener.PeerDisconnectedEvent += OnPeerDisconnected;
        _listener.NetworkReceiveEvent += OnNetworkReceive;
    }

    public void Start()
    {
        _manager.Start(7777);
        Console.WriteLine("[RUDP] Real-time Relay Server started on port 7777");
    }

    private void OnPeerConnected(NetPeer peer)
    {
        Console.WriteLine($"[RUDP] Client connected: {peer.EndPoint}");
    }
```

```
    private void OnPeerDisconnected(NetPeer peer, DisconnectInfo disconnectInfo)
    {
        Console.WriteLine($"[RUDP] Client disconnected: {peer.EndPoint}");
    }

    private void OnNetworkReceive(NetPeer fromPeer, NetPacketReader reader, byte
channel, DeliveryMethod method)
    {
        // Đọc PlayerState từ client
        var state = new PlayerState
        {
            PlayerId = reader.GetString(),
            Username = reader.GetString(),
            Position = new Vector3(reader.GetFloat(), reader.GetFloat(),
reader.GetFloat()),
            RotationY = reader.GetFloat()
        };

        Console.WriteLine($"[RUDP] Received from {state.Username} at
{state.Position}");

        // Relay cho tất cả client khác
        _writer.Reset();
        _writer.Put(state.PlayerId);
        _writer.Put(state.Username);
        _writer.Put(state.Position.X);
        _writer.Put(state.Position.Y);
        _writer.Put(state.Position.Z);
        _writer.Put(state.RotationY);

        foreach (var peer in _manager.ConnectedPeerList)
        {
            if (peer != fromPeer)
            {
                peer.Send(_writer, DeliveryMethod.Unreliable); // Position update
uses Unreliable
            }
        }
    }
```

```
    public void Stop()
    {
        _manager.Stop();
    }
}
```

Step 5: Integrate RUDP Service into Program.cs

Open Program.cs and update it as follows:

```
using GameInventoryApi.Repositories;
using GameInventoryApi.Services;
using MongoDB.Driver;
using Microsoft.Extensions.Options;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.Configure<MongoDbSettings>(builder.Configuration.GetSection("MongoDbSettings"));
builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("JwtSettings"));

builder.Services.AddSingleton<IMongoDatabase>(sp =>
{
    var settings = sp.GetRequiredService<IOptions<MongoDbSettings>>().Value;
    var client = new MongoClient(settings.ConnectionString);
    return client.GetDatabase(settings.DatabaseName);
});

builder.Services.AddScoped(typeof(IMongoRepository<>), typeof(MongoRepository<>));

builder.Services.AddScoped<IInventoryService, InventoryService>();
builder.Services.AddScoped<IPlayerProfileService, PlayerProfileService>();
builder.Services.AddScoped<IAuthService, AuthService>();

// === RUDP Real-time Relay Server ===
builder.Services.AddSingleton<RudpServerService>();

// JWT Authentication (đã hoàn chỉnh)
var jwtSettings =
```

```
builder.Configuration.GetSection("JwtSettings").Get<JwtSettings>()!;
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings.SecretKey))
        };
    });

builder.Services.AddAuthorization();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    // Copy nguyên code SwaggerGen từ Assignment 1 của bạn vào đây
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

// === KHỞI ĐỘNG RUDP SERVER ===
var rudpService = app.Services.GetRequiredService<RudpServerService>();
rudpService.Start();

Console.WriteLine("=== FULL GAME SERVER IS RUNNING ===");
Console.WriteLine("REST API     → https://localhost:5xxx/swagger");
Console.WriteLine("RUDP Real-time → port 7777");

app.Run();
```

Step 6: Run and Test the Full Server

1. Make sure MongoDB Docker container is running (from Assignment 1).
2. Run the project:

```
dotnet run
```

You should see:

```
REST API starting
"RUDP Real-time Relay Server started on port 7777"
```

Test REST API (as before) → https://localhost:5xxx/swagger

PART 2: CREATE TEST CLIENT APP

Step 1: Create the Test Client Project

1. Open a terminal in the folder that contains your GameInventoryApi project.
2. Run the following commands:

```
dotnet new console -o GameInventoryApi.TestClient
cd GameInventoryApi.TestClient
dotnet add package LiteNetLib --version 1.2.0
```

Step 2: Replace the entire content of Program.cs

Open the file GameInventoryApi.TestClient/Program.cs and **replace everything** with the code below:

GameInventoryApi.TestClient/Program.cs

```
using LiteNetLib;
using LiteNetLib.Utils;
using System.Numerics;
using System.Text;
using System;
using System.Threading.Tasks;

namespace GameInventoryApi.TestClient;

class Program
{
    private static NetManager? _client;
    private static readonly NetDataWriter _writer = new();
    private static readonly string _playerId = Guid.NewGuid().ToString();
    private static string _username = "Player" + new Random().Next(1000, 9999);
    private static Vector3 _position = new Vector3(0, 0, 0);
    private static float _rotationY = 0f;
    private static bool _running = true;

    static async Task Main(string[] args)
    {
        Console.Title = $"RUDP Test Client - {_username}";

        Console.WriteLine($"[CLIENT] Starting as {_username} (ID: {_playerId})");

        var listener = new EventBasedNetListener();
        _client = new NetManager(listener);
```

```
    // Register event handlers
    listener.NetworkReceiveEvent += OnNetworkReceive;
    listener.PeerConnectedEvent += OnPeerConnected;
    listener.PeerDisconnectedEvent += OnPeerDisconnected;

    _client.Start();
    _client.Connect("127.0.0.1", 7777, "");   // Connect to localhost:7777

    Console.WriteLine("[CLIENT] Connected to RUDP server on port 7777");
    Console.WriteLine("Press ESC to exit | Use WASD to move | Q/E to rotate");

    // Start sending position updates in background
    _ = Task.Run(SendPositionLoop);

    // Main input loop
    while (_running)
    {
        if (Console.KeyAvailable)
        {
            var key = Console.ReadKey(true).Key;
            if (key == ConsoleKey.Escape)
                _running = false;
            else
                HandleMovement(key);
        }

        await Task.Delay(16); // ~60 FPS input responsiveness
    }

    _client.Stop();
    Console.WriteLine("[CLIENT] Disconnected from server.");
}

/// <summary>
/// Sends player position to the server every 200ms (Unreliable)
/// </summary>
private static async Task SendPositionLoop()
{
    while (_running && _client != null)
    {
        SendPlayerState();
        await Task.Delay(200);
    }
}

private static void SendPlayerState()
{
    if (_client?.FirstPeer == null) return;

    _writer.Reset();
    _writer.Put(_playerId);
    _writer.Put(_username);
```

```
        _writer.Put(_position.X);
        _writer.Put(_position.Y);
        _writer.Put(_position.Z);
        _writer.Put(_rotationY);

        _client.FirstPeer.Send(_writer, DeliveryMethod.Unreliable);
    }

    private static void HandleMovement(ConsoleKey key)
    {
        const float speed = 0.5f;

        switch (key)
        {
            case ConsoleKey.W: _position.Z += speed; break;
            case ConsoleKey.S: _position.Z -= speed; break;
            case ConsoleKey.A: _position.X -= speed; break;
            case ConsoleKey.D: _position.X += speed; break;
            case ConsoleKey.Q: _rotationY -= 10f; break;
            case ConsoleKey.E: _rotationY += 10f; break;
        }
    }

    private static void OnPeerConnected(NetPeer peer)
    {
        Console.WriteLine($"[CLIENT] Successfully connected to server:
{peer.EndPoint}");
    }

    private static void OnPeerDisconnected(NetPeer peer, DisconnectInfo disconnectInfo)
    {
        Console.WriteLine($"[CLIENT] Disconnected from server. Reason:
{disconnectInfo.Reason}");
    }

    private static void OnNetworkReceive(NetPeer fromPeer, NetPacketReader reader, byte
channel, DeliveryMethod method)
    {
        try
        {
            var state = new PlayerState
            {
                PlayerId = reader.GetString(),
                Username = reader.GetString(),
                Position = new Vector3(reader.GetFloat(), reader.GetFloat(),
reader.GetFloat()),
                RotationY = reader.GetFloat()
            };

            Console.WriteLine($"[RECEIVED] {state.Username} @ ({state.Position.X:F1},
{state.Position.Y:F1}, {state.Position.Z:F1}) rot={state.RotationY:F0}°");
        }
        catch (Exception ex)
```

```
        {
            Console.WriteLine($"[ERROR] Failed to deserialize packet: {ex.Message}");
        }
    }
}

// PlayerState class (exact match with server for serialization)
public class PlayerState
{
    public string PlayerId { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public Vector3 Position { get; set; }
    public float RotationY { get; set; }
}
```

Step 3: Build and Run

```
cd GameInventoryApi.TestClient
dotnet build
```

Step 4: How to Test (Recommended for Assignment Submission)

1. Start your **GameInventoryApi** server first (dotnet run in the main project).
2. Open **2 or 3 separate terminals**.
3. In each terminal, navigate to the TestClient folder and run:

```
dotnet run
```

4. In each client window:
    - Use **WASD** to move
    - Use **Q / E** to rotate
    - Press **ESC** to quit

You should immediately see other players' positions being relayed in real time in every client console.

Step 5: Screenshots for README.md (Submission)

Take these screenshots:

- Server console showing “RUDP Real-time Relay Server started on port 7777”
- At least 2–3 client consoles running at the same time
- Clear [RECEIVED] messages showing other players’ movement

Submission Requirements

Submit **one ZIP file** containing:

1. Entire updated GameInventoryApi project
2. README.md with:
    - Screenshot of Swagger UI (REST API still working)
    - Screenshot of console showing RUDP server started + clients connecting
    - Screenshot showing real-time data being relayed between clients
    - Short explanation: “How does your server combine REST and RUDP?”

Grading Rubric (Total: 10.0 points)

No.	Criteria	Points	Description
1	LiteNetLib package added correctly	1.0	Project builds without errors
2	RudpServerService created with proper events	2.0	PeerConnected, NetworkReceive, PeerDisconnected handled
3	Real-time data relay (broadcast to other peers)	2.5	PlayerState is correctly sent and received
4	Proper use of DeliveryMethod	1.5	Unreliable for position, etc.
5	Integration with existing REST API	1.5	Both services run in the same project
6	Testing & Documentation (README + screenshots)	1.0	Clear evidence

No.	Criteria	Points	Description
Bonus	Create Test Client App	**+0.5**	Advanced